

مستند پروژه زمان‌بندی مسابقات ورزشی با CSP

۱. معرفی کلی پروژه

در این پروژه، مسئله زمان‌بندی هفتگی مسابقات ورزشی به صورت یک مسئله ارضای محدودیت یا CSP مدل‌سازی و حل شده است. هدف برنامه این است که برای هر مسابقه، یک روز، یک ساعت و یک استادیوم مشخص انتخاب کند؛ به شکلی که قوانین و محدودیت‌های مسئله نقض نشوند.

در این مدل، هر مسابقه یک متغیر در نظر گرفته می‌شود. دامنه هر متغیر شامل تمام حالت‌های ممکن برای برگزاری آن مسابقه است. هر حالت به صورت یک سه‌تایی شامل روز، ساعت و استادیوم تعریف می‌شود.

به عنوان مثال:

(day, hour, stadium)

یعنی مسابقه در یک روز مشخص، در یک ساعت مشخص و در یک استادیوم مشخص برگزار شود.

۲. مدل‌سازی مسئله به صورت CSP

در یک مسئله CSP سه بخش اصلی وجود دارد:

۱. متغیرها
۲. دامنه‌ها
۳. محدودیت‌ها

در این پروژه این سه بخش به شکل زیر تعریف شده‌اند:

متغیرها

هر مسابقه بین دو تیم یک متغیر است. برای مثال اگر مسابقه بین Team_A و Team_B باشد، این مسابقه یک متغیر جداگانه در مسئله محسوب می‌شود.

دامنه‌ها

دامنه هر مسابقه شامل همه ترکیب‌های ممکن از روز، ساعت و استادیوم است. اگر تعداد روزها D، تعداد ساعت‌ها در هر روز H و تعداد استادیوم‌ها S باشد، دامنه اولیه هر مسابقه شامل $S \times D \times H$ حالت ممکن خواهد بود.

محدودیت‌ها

برای اینکه برنامه مسابقات معتبر باشد، چند محدودیت اصلی بررسی می‌شود:

۱. در یک استادیوم، در یک روز و ساعت مشخص، فقط یک مسابقه می‌تواند برگزار شود.
۲. هر تیم در هر روز فقط می‌تواند یک مسابقه داشته باشد.
۳. هیچ دو مسابقه حساس نباید در یک روز برگزار شوند.

۳. الگوریتم استفاده شده

برای حل مسئله از الگوریتم Backtracking Search استفاده شده است. این الگوریتم به صورت مرحله به مرحله برای مسابقات مقدار انتخاب می‌کند. اگر در یک مرحله انتخاب انجام شده باعث بن بست شود، الگوریتم به مرحله قبل برمی‌گردد و مقدار دیگری را امتحان می‌کند.

در کنار Backtracking از Forward Checking نیز استفاده شده است. Forward Checking بعد از هر تخصیص، دامنه مسابقات باقی‌مانده را بررسی و مقدارهای ناسازگار را حذف می‌کند. اگر دامنه یک مسابقه خالی شود، یعنی ادامه دادن این مسیر ممکن نیست و الگوریتم باید Backtrack کند.

برای بهتر شدن عملکرد الگوریتم، دو روش MRV و LCV نیز استفاده شده‌اند.

۴. روش MRV

MRV مخفف Minimum Remaining Values است. در این روش، الگوریتم به جای اینکه همیشه اولین مسابقه تخصیص داده نشده را انتخاب کند، مسابقه‌ای را انتخاب می‌کند که کمترین تعداد مقدار ممکن در دامنه خود دارد.

دلیل استفاده از MRV این است که اگر یک مسابقه محدودیت بیشتری داشته باشد، بهتر است زودتر بررسی شود تا بن بست‌ها سریع‌تر شناسایی شوند. این کار باعث کاهش تعداد انتخاب‌های بی‌فایده و بهتر شدن عملکرد جستجو می‌شود.

در کد، این کار توسط تابع `select_unassigned` انجام می‌شود.

۵. روش LCV

LCV مخفف Least Constraining Value است. در این روش، الگوریتم مقدارهای ممکن برای یک مسابقه را به ترتیبی امتحان می‌کند که ابتدا مقدارهایی انتخاب شوند که کمترین محدودیت را برای مسابقات بعدی ایجاد می‌کنند.

یعنی اگر یک مقدار باعث شود گزینه‌های زیادی از دامنه مسابقات دیگر حذف شود، آن مقدار اولویت کمتری دارد. در مقابل، مقداری که کمترین حذف را ایجاد کند، زودتر امتحان می‌شود.

در کد، تابع `order_domain_values` مقدارها را مرتب می‌کند و تابع `count_removed_values` محاسبه می‌کند که هر مقدار چه تعداد گزینه را از دامنه مسابقات آینده حذف می‌کند.

۶. Forward Checking

Forward Checking یکی از بخش‌های مهم کد است. بعد از اینکه برای یک مسابقه روز، ساعت و استادیوم انتخاب شد، این تابع دامنه مسابقات باقی‌مانده را به‌روزرسانی می‌کند.

این تابع سه نوع مقدار نامعتبر را حذف می‌کند:

۱. اگر یک ترکیب روز، ساعت و استادیوم قبلاً استفاده شده باشد، آن ترکیب از دامنه مسابقات دیگر حذف می‌شود.
۲. اگر یکی از تیم‌های مسابقه در یک روز بازی داشته باشد، آن روز از دامنه بازی‌های دیگر همان تیم حذف می‌شود.
۳. اگر مسابقه فعلی حساس باشد، روز انتخاب‌شده برای سایر مسابقات حساس حذف می‌شود.

اگر بعد از این حذف‌ها دامنه یک مسابقه خالی شود، تابع مقدار False برمی‌گرداند و الگوریتم وارد مرحله Backtracking می‌شود.

۷. بررسی سازگاری

قبل از اینکه یک مقدار به یک مسابقه اختصاص داده شود، باید بررسی شود که این مقدار با مسابقات قبلاً زمان‌بندی‌شده تداخل نداشته باشد. این بررسی در تابع consistent انجام می‌شود.

این تابع سه مورد را بررسی می‌کند:

۱. آیا در همان روز، ساعت و استادیوم قبلاً مسابقه‌ای ثبت شده است یا نه.
۲. آیا یکی از تیم‌های مسابقه در همان روز بازی دیگری دارد یا نه.
۳. آیا دو مسابقه حساس در یک روز قرار گرفته‌اند یا نه.

اگر هرکدام از این محدودیت‌ها نقض شود، تابع مقدار False برمی‌گرداند. در غیر این صورت مقدار انتخاب‌شده قابل قبول است.

۸. بررسی اولیه امکان‌پذیری

قبل از شروع الگوریتم جستجو، برنامه دو بررسی ساده انجام می‌دهد تا اگر مسئله از ابتدا غیرقابل حل باشد، بدون جستجوی اضافی نتیجه No Solution چاپ شود.

بررسی اول مربوط به ظرفیت کل است. اگر تعداد مسابقات بیشتر از تعداد کل ظرفیت‌های ممکن باشد، مسئله جواب ندارد.

فرمول ظرفیت کل به شکل زیر است:

$$S \times D \times H$$

که در آن:

S تعداد استادیوم‌ها

D تعداد روزها

H تعداد ساعت‌های هر روز

اگر $S \times D \times H < N$ باشد، برنامه مستقیماً No Solution چاپ می‌کند.

بررسی دوم مربوط به مسابقات حساس است. چون هیچ دو مسابقه حساس نباید در یک روز باشند، اگر تعداد مسابقات حساس از تعداد روزها بیشتر باشد، مسئله جواب ندارد. یعنی اگر $D < K$ باشد، خروجی No Solution خواهد بود.

۹. توضیح توابع اصلی کد

تابع backtracking

این تابع هسته اصلی الگوریتم است. ابتدا بررسی می‌کند که آیا همه مسابقات زمان‌بندی شده‌اند یا نه. اگر همه مسابقات مقدار گرفته باشند، جواب نهایی برگردانده می‌شود.

در غیر این صورت، ابتدا با استفاده از MRV یک مسابقه انتخاب می‌شود. سپس مقدارهای ممکن آن مسابقه با استفاده از LCV مرتب می‌شوند. بعد الگوریتم هر مقدار را بررسی می‌کند. اگر مقدار با تخصیص‌های قبلی سازگار باشد، آن مقدار موقتاً به مسابقه اختصاص داده می‌شود و سپس Forward Checking اجرا می‌شود.

اگر مسیر انتخاب‌شده به جواب برسد، نتیجه برگردانده می‌شود. اگر هیچ مقداری جواب ندهد، شمارنده Backtracks افزایش پیدا می‌کند و تابع مقدار None برمی‌گرداند.

تابع select_unassigned

این تابع مسابقه بعدی را برای تخصیص انتخاب می‌کند. در این تابع از روش MRV استفاده شده است. یعنی بین مسابقاتی که هنوز زمان‌بندی نشده‌اند، مسابقه‌ای انتخاب می‌شود که کوچکترین دامنه را دارد.

تابع order_domain_values

این تابع مقدارهای ممکن برای یک مسابقه را مرتب می‌کند. مرتب‌سازی بر اساس روش LCV انجام می‌شود. یعنی مقدارهایی که محدودیت کمتری برای مسابقات آینده ایجاد می‌کنند، زودتر امتحان می‌شوند.

تابع count_removed_values

این تابع برای یک مقدار مشخص حساب می‌کند که اگر آن مقدار به مسابقه فعلی داده شود، چند مقدار از دامنه مسابقات دیگر حذف خواهد شد. خروجی این تابع در روش LCV استفاده می‌شود.

تابع consistent

این تابع بررسی می‌کند که آیا انتخاب یک روز، ساعت و استادیوم برای مسابقه فعلی با مسابقات قبلی سازگار است یا نه. در این تابع تداخل استادیوم، تداخل تیم‌ها در یک روز و تداخل مسابقات حساس بررسی می‌شود.

تابع forward_check

این تابع بعد از هر تخصیص اجرا می‌شود و دامنه مسابقات باقی‌مانده را محدودتر می‌کند. اگر در اثر حذف مقدارهای ناسازگار، دامنه یک مسابقه خالی شود، تابع مقدار False برمی‌گرداند و الگوریتم به عقب برمی‌گردد.

تابع main

تابع main ورودی‌ها را از کاربر می‌خواند، لیست مسابقات و مسابقات حساس را آماده می‌کند، بررسی‌های اولیه امکان‌پذیری را انجام می‌دهد، دامنه اولیه مسابقات را می‌سازد و سپس الگوریتم backtracking را اجرا می‌کند.

در پایان، اگر جواب پیدا شود، برنامه مسابقات را چاپ می‌کند. در غیر این صورت عبارت No Solution نمایش داده می‌شود. همچنین تعداد Backtrack‌ها و زمان اجرای برنامه نیز چاپ می‌شود.

۱۰. فرمت ورودی

ورودی برنامه به ترتیب شامل موارد زیر است:

۱. تعداد استادیوم‌ها
۲. نام استادیوم‌ها
۳. تعداد روزها
۴. تعداد ساعت‌ها در هر روز
۵. تعداد مسابقات
۶. لیست مسابقات
۷. تعداد مسابقات حساس
۸. لیست مسابقات حساس

نمونه ورودی:

```
2
Azadi Takhti
2
2
4
Team_A Team_B
Team_C Team_D
Team_A Team_E
Team_B Team_C
0
```

۱۱. فرمت خروجی

اگر برنامه بتواند یک زمان‌بندی معتبر پیدا کند، برای هر مسابقه خروجی زیر چاپ می‌شود:

Team1 Team2 Day Hour Stadium

سپس تعداد Backtrack‌ها و زمان اجرا چاپ می‌شود.

نمونه خروجی:

Team_A Team_B 1 1 Azadi
Team_C Team_D 1 1 Takhti
Team_A Team_E 2 1 Azadi
Team_B Team_C 2 1 Takhti
Backtracks: 0
Time: 0.0012 seconds

اگر مسئله جواب نداشته باشد، خروجی به شکل زیر خواهد بود:

No Solution
Backtracks: X
Time: Y seconds

۱۲. تعریف Backtrack در این پروژه

در این کد، هر زمانی که الگوریتم برای یک مسابقه همه مقدارهای ممکن را امتحان کند اما هیچ‌کدام به جواب معتبر نرسند، یک Backtrack ثبت می‌شود.

بنابراین مقدار Backtracks نشان می‌دهد الگوریتم چند بار به بن‌بست رسیده و مجبور شده است به مرحله قبل برگردد.

۱۳. جمع‌بندی

در این پروژه، مسئله زمان‌بندی مسابقات ورزشی با استفاده از مدل CSP حل شده است. برای حل مسئله از Backtracking Search استفاده شده و برای بهبود عملکرد آن Forward Checking، MRV و LCV به کار رفته‌اند.

کد ابتدا شرایط غیرممکن را بررسی می‌کند تا از جستجوی بی‌فایده جلوگیری شود. سپس با ساخت دامنه اولیه برای هر مسابقه، تلاش می‌کند برای همه مسابقات یک زمان و استادیوم معتبر پیدا کند.

این پیاده‌سازی علاوه بر محدودیت‌های اصلی مسئله، محدودیت مسابقات حساس را نیز پشتیبانی می‌کند و در پایان تعداد Backtrack‌ها و زمان اجرای برنامه را نمایش می‌دهد.